RESEARCH PAPER

Testing What a Score Measures: Validating and Repairing the Calibration Scores of a Personal Decision Analytics System

Shakil Ur Rehman*

Department of Computer Science and Engineering, Noida Institute of Engineering and Technology, Greater Noida, India

Running title: Testing what a score measures

***Correspondence:** Shakil Ur Rehman, Department of Computer Science and Engineering, Noida Institute of Engineering and Technology, 19, Institutional Area, Knowledge Park II, Greater Noida 201306, Uttar Pradesh, India. Email: rehmanshakil21@gmail.com. ORCID: 0009-0004-3050-0054

Data availability statement. The raw results of every experiment, the derived ForecastBench dataset (CC BY-SA 4.0) and the script that derives it are openly available at `https://github.com/ShakilUrRehman21/digital-memory-vault-paper` and are provided as Supporting Information S1. The original ForecastBench data are available at `https://github.com/forecastingresearch/forecastbench-datasets`.

Code availability. The source code of the baseline and revised implementations and the evaluation kit, which builds both implementations and reproduces every result with one command, are available under the MIT licence in the same repository.

Funding statement. This research received no specific grant from any funding agency in the public, commercial or not-for-profit sectors.

Conflict of interest statement. The author developed the software evaluated in this study and has no other conflict of interest to declare.

Ethics approval statement. Not applicable. The study involved no human participants. The only data about people are pseudonymous forecasts published by the Forecasting Research Institute under the Creative Commons Attribution-ShareAlike 4.0 licence, used as published.

Patient consent statement. Not applicable.

Permission to reproduce material from other sources. No previously published figures or tables are reproduced. The ForecastBench forecasts are used and redistributed in derived form under their CC BY-SA 4.0 licence, with attribution.

Clinical trial registration. Not applicable.

Author contributions. The sole author conceived the study, developed the software, designed the evaluation, analysed the results and prepared the manuscript, with the assistance described in Section 4.6.

Abstract

Self-tracking applications turn what users record into scores, yet whether a score measures what its name claims is rarely tested, because no oracle gives the correct score for a real person. We show how such scoring code can be tested without human participants and apply the approach to Digital Memory Vault, an open-source application that records personal decisions as predictions and scores them against their outcomes. Three oracles replace the missing ground truth: metamorphic relations derived from each score's name, synthetic users whose miscalibration is fixed by the generator, and 4,793 resolved forecasts by 39 superforecasters compared with standard calibration measures. A differential check ties them to the running application; mutation, end-to-end, load and network-latency tests complete the evaluation. In the application's baseline implementation, risk calibration violated every calibration relation on every input, confidence calibration could not detect risk-dependent errors at any history length, and the emotional-bias score counted negated words; the tests also exposed editable predictions, a race that stored two outcomes for one decision, unescaped report fields and one query per decision. In the revised implementation all tests pass, the calibration scores track reference measures on real forecasts (Spearman $\rho = -0.91$ and -0.96, against -0.42 and -0.08 for the baseline rules), 20 decisions suffice in simulation to flag two-step miscalibration, and analytics latency at 1,000 decisions falls from 924.1 to 36.7 ms. Scoring rules are measurement instruments and need the same testing as the code that computes them.

Keywords: metamorphic testing; mutation testing; test oracle; measurement validity; confidence calibration; personal informatics

1 Introduction

Self-tracking applications rarely show users their raw records. They show scores: a sleep score, a readiness score, a focus score. A score is a measurement instrument, and a user who trusts it will draw conclusions from it. If the score does not measure what its name says, the user learns the wrong lesson, and nothing in the interface reveals the problem. Yet the testing that such applications receive usually checks that the code computes its formula, not that the formula measures the attribute it is named after. The second question has no obvious test oracle: there is no ground truth for how well calibrated a particular person is, so the expected value of the score is unknown [1]. The common response is to defer validity to a user study, which is expensive, slow and itself depends on a valid reference measure.

This paper argues that much of score validity can be tested without participants, by borrowing three ideas. Metamorphic testing replaces the missing oracle with relations between inputs and outputs [2, 3, 4]; for a score, the relations follow from its name. A calibration score, for instance, must fall when every stated confidence is raised by two steps. Simulation with a known generating model supplies ground truth by construction, so one can measure how often a score detects a miscalibration that is known to be there, and after how many records. Public data from real forecasters, finally, can be passed through the scoring code and compared with calibration measures that the forecasting literature has used for decades [5, 6].

We apply these ideas to Digital Memory Vault, an open-source web application in which a user records each consequential decision as a prediction (stated confidence, emotional state, risk level and expected outcome) before the result is known, later adds a success rating and a reflection, and receives four scores: decision accuracy, confidence calibration, risk calibration and emotional bias. Decisions of this kind seldom produce feedback, people are often poorly

calibrated about them, and calibration improves with feedback [7, 8, 9], so the application is only useful if its scores are right. We first evaluate the application's baseline implementation, whose scoring rules follow the simplest reading of each score's name (compare average confidence with average success; average the success of risky and safe decisions; look for warning words in reflections), then correct every defect the evaluation finds and evaluate the resulting revised implementation with the same instruments. Both implementations are included in the published artefact and the evaluation builds them from source, so every comparison in this paper is between two runs of the same experiments.

The paper makes four contributions:

- A validation approach for scoring code that needs no human participants: metamorphic relations derived from score names, simulation with known miscalibration including the history length each score needs, and replay of public real forecasts against established reference measures, tied to the running application by a differential check.
- A case study applying it, together with functional, concurrency, security and performance tests, to a complete application. It found two invalid scores, defects in the text matching of a third, four integrity and security defects, a query anti-pattern and unbounded snapshot growth.
- A revised implementation whose scoring rules are specified exactly and verified by the same instruments, including a measured minimum history below which the application now marks its scores as provisional.
- An evaluation kit that builds both implementations from source and reproduces every result with one command, released with the derived real-forecast dataset.

The evaluation is organised around four research questions (RQs):

- **RQ1 (integrity and security).** Does the implementation protect the decision record, that is, predictions that cannot be revised after the fact, one outcome per decision, safe output and per-user access, and do the revisions close the gaps found?
- **RQ2 (performance).** How do latency and throughput change with the length of a user's history and with the round-trip time to the database?
- **RQ3 (score validity).** Do the scores respond to the behaviour they are named after, under metamorphic relations, for synthetic users with known behaviour and on real forecasts, and how many decisions do they need?
- **RQ4 (test adequacy).** How well do example-based, metamorphic and property-based tests detect faults injected into the scoring code?

Section 2 reviews related work and Section 3 describes the system and its baseline and revised scoring rules. Section 4 sets out the evaluation method, Section 5 the revisions, and Section 6 the results. Section 7 discusses them, Section 8 examines threats to validity and Section 9 concludes.

2 Background and related work

2.1 Calibration and how it is measured

A forecaster is well calibrated if, among the occasions on which they say they are 70% sure, they are right about 70% of the time. People usually fall short of this, and the typical direction of error is overconfidence [7, 8]. Calibration can be trained: in a multi-year geopolitical forecasting tournament, training and teamwork reduced overconfidence to about one percentage point [10, 9].

The measurement tools come mostly from weather forecasting. The Brier score averages the squared difference between probability forecasts and binary outcomes [5], and Murphy's decomposition splits it into a reliability term (calibration), a resolution term and the uncertainty of the outcomes [6]. A reliability diagram groups forecasts by stated probability and compares each group's mean forecast with its observed frequency; the expected calibration error (ECE) summarises it as the weighted mean absolute gap across groups [11, 12]. Two simpler summaries are common and weaker. Calibration-in-the-large compares the overall mean forecast with the overall frequency, so errors in opposite directions cancel. Calibration within groups, used in the fairness literature, requires the same agreement inside every subgroup of interest [13]. The baseline rules of the application measure only calibration-in-the-large, and Section 6.3 shows the consequence.

2.2 Decision journals and personal informatics

Personal informatics tools help people collect and reflect on data about themselves [14], but a mapping review of 523 publications found that most of the field concerns health and wellness [15]. Decision journals rarely connect a stated confidence to a later outcome. Doing so guards against hindsight bias, the tendency to believe after the fact that an outcome was predictable [16], but only if the prediction cannot be edited later. Outcome bias, judging a decision by its result [17], remains, and emotions at the moment of choice shift risk judgments in systematic ways [18]. None of this literature tests the scoring software that would deliver such feedback.

2.3 Validity of software measures

Software engineering has long asked whether its own measures measure what they claim. Kitchenham et al. proposed a framework in which a valid measure must preserve the empirical relations of the attribute it measures [19]; Briand et al. stated such requirements as mathematical properties a measure must satisfy, for example that merging unrelated modules must not reduce a size measure [20]; and Kaner and Bond argued that many metrics in use had never been checked against the attribute they name [21]. The metamorphic relations in Section 4.4.1 apply the same idea to scores computed from personal data: each relation is a property that any measure deserving the score's name must have.

2.4 Testing without an oracle

When the expected output of a program is unknown, testing faces the oracle problem [1]. Metamorphic testing addresses it by checking relations among the outputs of related inputs instead of individual outputs [2, 3, 4], and has been used for machine learning classifiers, whose correct output is likewise unknown [22]. Property-based testing checks invariants on many generated inputs [23]. Mutation analysis measures how good a test set is by the share of injected faults

(mutants) it detects [24, 25]. We use all three: metamorphic relations both as an evaluation instrument and as executable tests, property-based tests for invariants of the scoring module, and mutation analysis to compare the fault-detection power of the different kinds of test.

2.5 Integrity and performance of database-backed web applications

Object-relational mapping (ORM) frameworks hide the SQL they generate, which makes it easy to issue one query per record where a single query would do; detectors for such anti-patterns and studies of open-source applications report large speed-ups from fixing them, often with small code changes [26, 27]. Check-then-act sequences in web applications are vulnerable to concurrent requests that interleave between the check and the action, a class of defects that Warszawski and Bailis found widespread in database-backed applications [28]. Unescaped user input in generated documents is an injection weakness in the OWASP Top 10 [29].

3 The system

3.1 Overview

Digital Memory Vault is a TypeScript application on Next.js with a server-rendered dashboard and PostgreSQL storage accessed through Drizzle ORM (Fig. 1). A proxy verifies the login token on every protected route and passes the user's identifier to the route handlers, which include it in every query, so a request for another user's record returns status 404. A decision is logged with a title, category, description, expected outcome, confidence (1–10), emotional state (calm, stressed, excited, pressured or uncertain), risk level (low, medium or high) and decision date. It stays pending until the user records an outcome: what happened, a success rating (1–10), a lesson and a reflection. Each decision can be exported as an HTML report. Whenever the analytics view is opened, the four scores are stored as a snapshot.

3.2 Scoring rules

Let R be the set of a user's resolved decisions, $n = |R|$, and for decision i let c_i be the stated confidence and s_i the success rating, both integers from 1 to 10. Bars denote means over the set indicated. Table 1 defines the four scores under the baseline and the revised rules; all are reported on a 0–100 scale, with ten points per step of error.

E_c is the expected calibration error of a reliability diagram whose bins are the ten confidence values, expressed in steps [11], and E_r is the corresponding calibration error within risk levels [13]. By the triangle inequality both are at least $|\bar{c} - \bar{s}|$, so the revised rules never rate a user as better calibrated than the baseline rules do. The twelve warning words are rushed, pressured, anxious, overconfident, doubt, reactive, panic, fear, desperate, impulsive, hasty and regret. In the revised implementation the rules live in a pure module without input or output, which the API, the tests and every offline experiment import, so the code that is evaluated is the code that runs.

Table 2 applies both sets of rules to ten illustrative decisions entered through the application. The two high-risk decisions were stated with confidence 9 and 8 and rated 3 and 4. The baseline risk calibration score of 56 does not register this mismatch, because it never reads confidence; the revised score is 85 and the revised implementation reports a gap of five steps in the high-risk group. The baseline confidence calibration score of 91 reflects only that mean confidence and

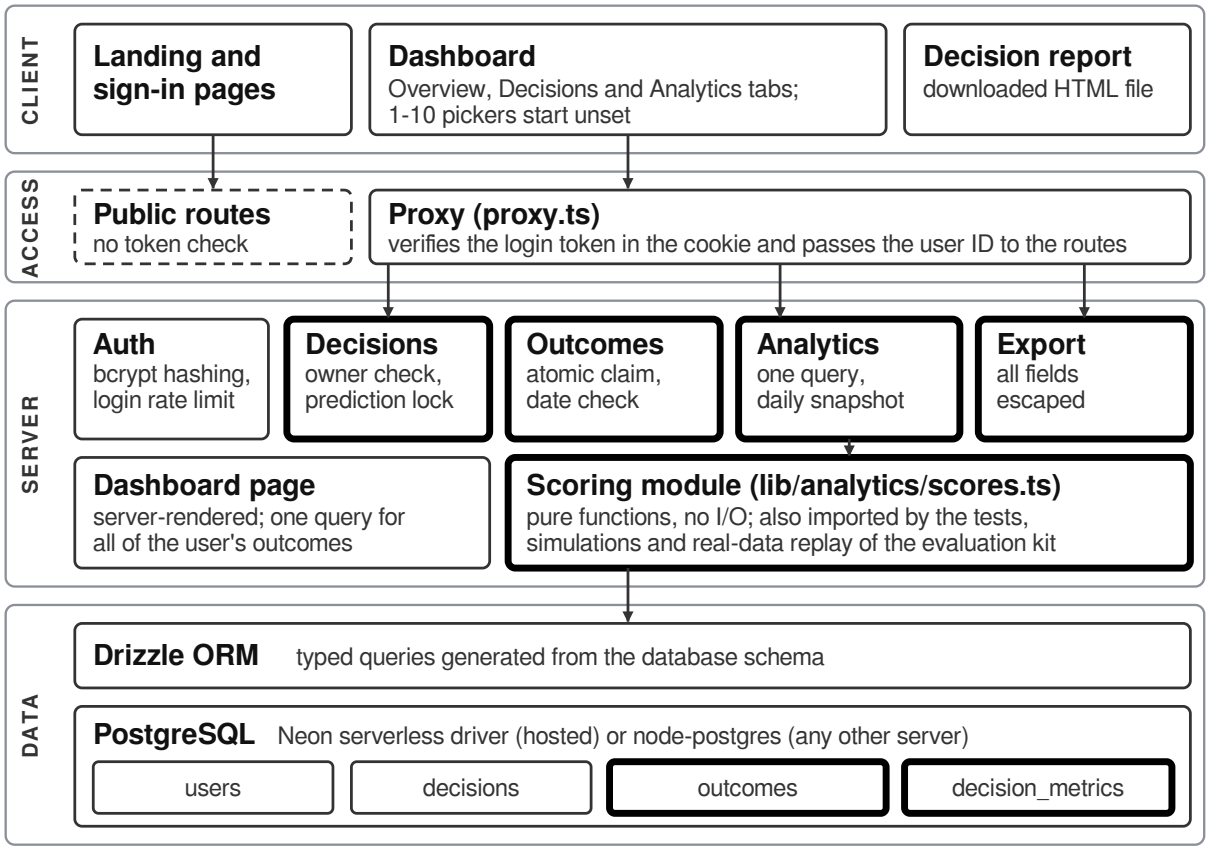


Figure 1. Architecture of Digital Memory Vault (revised implementation). Boxes show components in the client, access, server and data layers, and arrows show the direction of requests. Components with a heavier outline differ from the baseline implementation; the dashed box marks public routes, which bypass the token check

mean success are close, although four decisions were overconfident and two underconfident by more than one step.

4 Evaluation method

Figure 2 summarises the approach. Each kind of evidence comes with its own oracle, the thing that decides whether a result is right. The scoring experiments run on the scoring rules as pure functions; a differential check connects them to the running application.

4.1 Implementations and environment

The artefact contains both implementations as source: the baseline (Next.js 16.1.6) and the revised implementation (Next.js 16.3.8). A script copies each, installs its own locked dependencies, creates a fresh database, builds the application with `next build` and starts it with `next start`. All measurements were taken in one run on a Linux machine (Ubuntu 24.04) with two virtual CPUs (Intel Xeon, 2.8 GHz), 7.8 GB of memory, Node.js 22.22 and PostgreSQL 16.15. The application servers and the database were pinned to the first CPU and the test clients to the second. The hosted Neon database used in production is not reachable from an isolated machine, so both implementations used the node-postgres driver against the local server; the revised implementation supports both drivers, while the baseline needed a seven-line substitute

Table 1. Baseline and revised scoring rules

Score	Baseline rules	Revised rules
Decision accuracy	$\frac{1}{n}\sum_{i\in R}\max(0,100-10\lvert c_i-s_i\rvert)$	unchanged
Confidence calibration	$\max(0,100-10\lvert\bar{c}-\bar{s}\rvert)$: compares two overall means	$\max(0,100-10E_c)$, $E_c=\sum_k\frac{n_k}{n}\lvert\bar{c}_k-\bar{s}_k\rvert$ over groups k of equal stated confidence
Risk calibration	mean of $10\bar{s}_{\text{high}}$ and $10\bar{s}_{\text{low}}$; confidence and medium risk ignored	$\max(0,100-10E_r)$, $E_r=\sum_g\frac{n_g}{n}\lvert\bar{c}_g-\bar{s}_g\rvert$ over $g\in\{\text{low, medium, high}\}$
Emotional bias	per decision, pending included: 40 if stressed, pressured or uncertain, plus 10 per warning word found as a substring of the reflection, capped at 100; mean over decisions	as baseline, but excited also scores 40, words match whole words and inflections only, and a word preceded within three words by a negation is ignored
Supporting values	over- and underconfidence rates: share of decisions with $c_i>s_i+1$ and $c_i<s_i-1$	unchanged; also signed gap $\bar{c}-\bar{s}$, E_c, E_r, gap per risk level, and a flag when $n<20$

c_i, stated confidence of decision i; s_i, its success rating; n, number of resolved decisions; E_c and E_r, calibration errors in steps of the 1–10 scale.

client file. Section 4.3 emulates network round trips explicitly.

4.2 Functional and safeguard tests

A Node.js script sends real HTTP requests and compares each response, and where needed the database state, with a predefined expectation. Tests TC01–TC33 cover authentication, route protection, access control between two users, decision logging, outcome recording, analytics and report export. Safeguard tests TC34–TC46 check that each prediction field is locked once an outcome exists, that an outcome cannot be dated before its decision was logged, that ten simultaneous outcome submissions for one decision store exactly one outcome, that every user-supplied field of the report is escaped, that one user cannot edit another's decision, that repeated analytics requests on one day store at most one snapshot, and three scoring relations from Section 4.4.1 checked through the API.

4.3 Performance

For each endpoint we measured the response time at the client over 30 sequential requests (20 for registration and login) and report the median and 95th percentile. To see how latency scales, we created users holding 10, 50, 100, 250, 500 and 1,000 resolved decisions, inserted with one SQL statement per table, and measured the analytics endpoint and the dashboard page over 30 requests each after a warm-up request. A small TCP proxy between application and database counted the SQL statements behind each request by parsing the PostgreSQL wire protocol, and could delay every packet in each direction to emulate a round-trip time (RTT) of 0, 2, 5, 10 or 20 ms. With all four application instances and both proxies running, responses through the proxy were slower than direct ones by a further 7 to 28 ms at zero added delay, more than the proxy adds in isolation (about 0.1 ms per query); we therefore compare the implementations by how their latency increases with RTT, not by absolute values. Under load, ten concurrent clients

Table 2. Worked example: ten illustrative decisions (not user data) and the scores returned by the application under the baseline and the revised rules

#	Decision	Risk	State	Confidence	Success
1	Accept internship offer	medium	calm	8	9
2	Buy a laptop on instalments	low	excited	7	7
3	Launch feature without user testing	high	pressured	9	3
4	Join a morning gym batch	low	calm	6	8
5	Move to a shared flat	medium	uncertain	7	5
6	Switch database provider mid-sprint	high	stressed	8	4
7	Apply for a hackathon	medium	calm	5	8
8	Invest savings in an index fund	medium	calm	6	6
9	Hire a freelance designer	medium	excited	9	5
10	Start a weekend course	low	calm	7	8

Rules	DA	CC	RC	EB	Over (%)	Under (%)
Baseline	77	91	56	18	40	20
Revised	77	81	85	26	40	20

DA, decision accuracy; CC, confidence calibration; RC, risk calibration; EB, emotional bias; Over and Under, over- and underconfidence rates. Reflections of decisions 3, 5, 6 and 9 contain warning words (“rushed”; “doubt”; “panic”, “hasty”; “impulsive”, “regret”). Confidence and success are on 1–10 scales.

requested the analytics endpoint for 20 seconds for a user with 100 and with 500 decisions. Finally, we audited production dependencies with `npm audit`.

4.4 Score validity

4.4.1 Metamorphic relations

Table 3 lists the relations. MR1–MR3 express what a calibration score must do when confidence is distorted in a known way; MR4–MR6 are invariances that any score computed from a set of independent decisions must have; MR7–MR9 concern the emotional-bias score’s text matching. Source histories were generated with 12 to 60 decisions in which confidence equalled the success rating, so the follow-up distortion was the only source of miscalibration. Each relation was checked on 1,000 generated source histories for the baseline and the revised rules, and we report the share of sources on which it was violated.

4.4.2 Synthetic users with known behaviour

Every synthetic decision has an underlying quality q drawn uniformly from 6–9 for low-risk, 4–7 for medium-risk and 2–6 for high-risk decisions; the success rating is q plus normal noise with standard deviation 1.5, rounded and kept within 1–10, and risk levels cycle through low, medium and high. We used two families of reporters. The first, the noisy-reporter design, states as confidence q plus a bias plus normal noise with standard deviation 1, with a calibrated, an overconfident ($+2$), an underconfident (-2) and an inconsistent profile (-3 on low-risk and $+3$ on high-risk decisions), 25 users of 40 decisions each, entered through the API of each implementation. Because of the added noise, that “calibrated” profile is calibrated on average only.

For the power analysis we defined a calibrated reporter as one who states the rounded

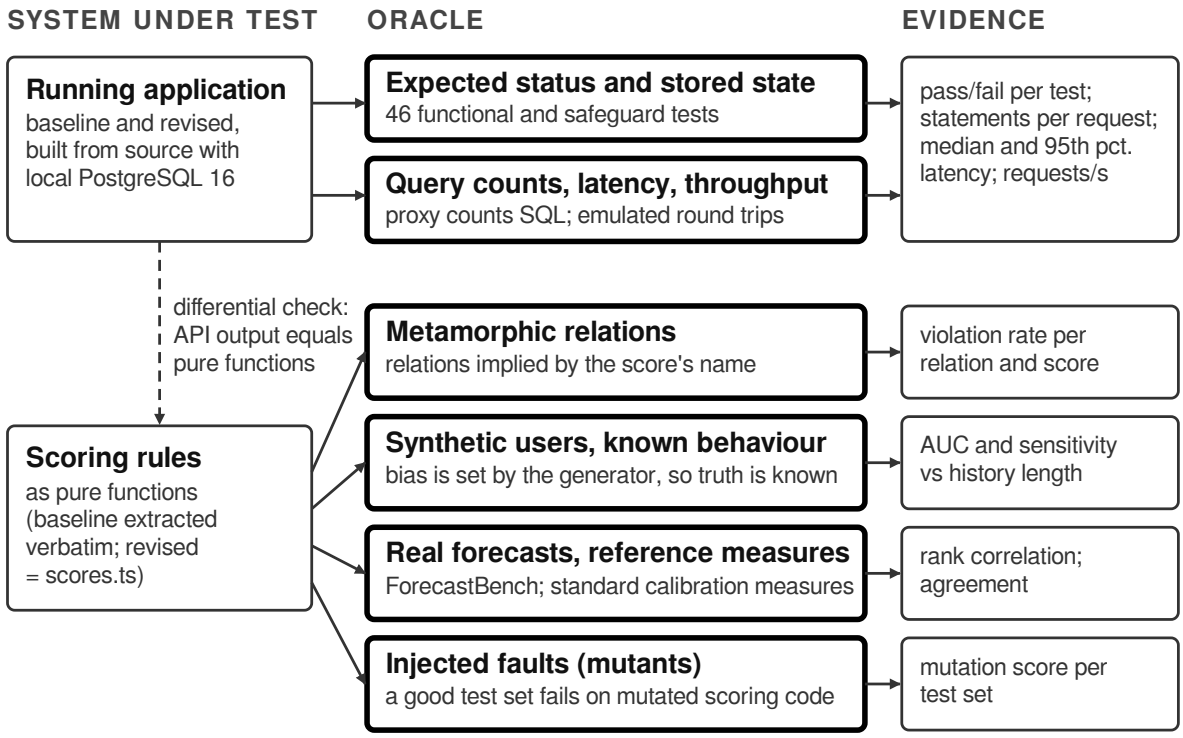


Figure 2. Validation approach. Left: the two forms in which each implementation is tested. Middle: the oracle that decides each verdict. Right: the evidence each produces. The dashed arrow is the differential check, which confirms that the pure scoring functions return exactly what the running application returns

quality, so that the mean success at each confidence level equals that confidence apart from clamping, and six miscalibrated profiles relative to it: overconfident by one and by two steps, underconfident by two steps, inconsistent by ± 2 and ± 3 steps by risk level, and noisy, with case-to-case noise of standard deviation 2 and no bias. Users with the same seed share identical outcomes and differ only in reported confidence. For each history length from 5 to 120 decisions and each profile we generated 1,000 miscalibrated and 1,000 calibrated users and computed, for each score, the area under the ROC curve (AUC) for separating the two groups [30] and the sensitivity at 95% specificity: the share of miscalibrated users scoring at or below the highest threshold that flags no more than 5% of calibrated users. We define the minimum history as the shortest length at which every two-step profile is flagged with at least 80% sensitivity by at least one revised calibration score.

4.4.3 Real forecasts

ForecastBench is a forecasting benchmark that publishes individual forecasts by human superforecasters together with resolutions [31]. From its question set of 21 July 2024 we took every individual superforecaster forecast whose question had resolved to 0 or 1: 4,793 forecasts by 39 forecasters with at least 20 resolved forecasts each (median 88, range 40–578); 28.3% of the questions resolved positively. A probability p was mapped to the application's scale as $c = 1 + \mathrm{round}(9p)$ and an outcome to a success rating of 10 or 1, so that a forecaster calibrated in probability terms is also calibrated on the 1–10 scale. The question's domain (prediction market, economic data series or event data series) took the place of the risk level as the grouping

Table 3. Metamorphic relations. A source history is transformed into a follow-up history, and the scores of the two must satisfy the stated relation

ID	Transformation of the source history	Required relation
MR1	Add 2 to every confidence	both calibration scores fall
MR2a	Subtract 2 from low-risk and add 2 to high-risk confidences (risk levels mixed at random)	both calibration scores fall
MR2b	As MR2a on histories with equal numbers of low- and high-risk decisions, where the two errors cancel in the overall mean	both calibration scores fall
MR3	Shift every confidence by $+3$ instead of $+1$	both calibration scores lower than with $+1$
MR4	Shuffle the order of decisions	all scores unchanged
MR5	Duplicate the whole history	all scores unchanged
MR6	Add pending decisions	accuracy and calibration unchanged
MR7	Append “No regrets, not anxious.” to every reflection	emotional bias does not rise
MR8	Append “Fearless and undoubtedly sound.”	emotional bias does not rise
MR9	Replace neutral reflections of calm decisions with “Panicked and regretted it.”	emotional bias rises

MR, metamorphic relation.

variable, since it is known when the forecast is made.

For each forecaster we computed the baseline and revised scores and four reference measures from the raw probabilities: ECE over ten equal-width bins [11], Murphy’s reliability term [6], calibration-in-the-large, and the domain-conditional calibration error, the weighted mean absolute gap between mean forecast and outcome frequency within each domain. We report Spearman rank correlations across forecasters with 95% bootstrap confidence intervals from 2,000 resamples [32]. We then distorted each forecaster’s real forecasts in four controlled ways (uniformly $+0.2$ and -0.2, -0.2 on market and $+0.2$ on event questions, and doubling the log-odds) and counted the forecasters for whom the score moved in the direction opposite to the reference error, that is, fell when the forecaster became less calibrated. Finally, for the 24 forecasters with at least 80 forecasts, we drew 200 random subsets of 10, 20, 40 and 80 forecasts and correlated the score on the subset with the reference measure on the full history.

4.4.4 Differential check

The offline experiments use the scoring rules as pure functions: for the revised rules the module the application imports, and for the baseline rules a verbatim extraction from the baseline implementation. To confirm that these functions describe the running applications, the noisy-reporter simulation and a ten-decision worked example (Table 2) were entered through each implementation’s API, and every score the API returned was compared with the pure function of the same rules on the same inputs.

4.5 Test adequacy

We mutated the revised scoring module with Stryker 9 [33] and ran five test sets against every mutant: a worked-example check of the kind developers typically write (the eight values reported for the ten decisions of Table 2, plus an empty history); the same with the metamorphic relations of Table 3 written as executable tests using fast-check [34]; the same with ten property-based tests of invariants such as bounds, perfect calibration giving 100, and determinism; that set with example-based unit tests added; and the example-based tests alone. Generated tests ran 100 cases each. Surviving mutants were inspected by hand for equivalence.

4.6 Use of generative artificial intelligence

A large language model (Claude, Anthropic) was used to help write the evaluation scripts, the code revisions and the code that renders the figures, and to help draft and edit the text. All scripts were run in the environment described above; the author reviewed the code, its raw output and the complete text, and takes full responsibility for the content. No figure was produced by a generative image model: diagrams are drawn from code and charts are plotted from measured data.

5 Revisions

Table 4 lists every defect the evaluation found in the baseline implementation, the revision made and the evidence used to verify it. Most revisions are small. Prediction locking compares the request with five protected fields and makes the update conditional on the decision still being pending, so the check and the write are atomic. Outcome recording first claims the decision with a single conditional update from pending to resolved, which only one concurrent request can win, and a unique index on the outcome's decision identifier enforces one outcome per decision in the database itself. Analytics and the dashboard read decisions and outcomes with one joined query, and snapshots are upserted on a unique key of user and day. The interface was revised too: the 1–10 ratings now start unset instead of at 7, to avoid anchoring [35], the outcome form asks optionally for a separate rating of the decision's quality, and the analytics view marks the calibration scores as provisional below 20 resolved decisions.

6 Results

6.1 RQ1: Integrity and security

The baseline passed 31 of the 33 core tests and 2 of the 13 safeguard tests; the revised implementation passed all 46 (Table 5). The failures in the baseline all concern the integrity of the record rather than crashes. The server accepted changes to every prediction field after the outcome was known, so hindsight could rewrite a prediction [16]. It also accepted an outcome dated years before the decision was logged. When ten outcome submissions for one decision arrived together, two were accepted and two outcomes were stored, because both requests passed the pending check before either recorded the outcome [28]. All six user-supplied text fields of the report were inserted unescaped, an injection weakness [29], and every visit to the analytics view added a snapshot row. Access control held in both implementations: every attempt by one user to read, edit, delete, export or resolve another user's decision returned 404.

Table 4. Defects found in the baseline implementation, revisions and how each was verified

Finding in the baseline	Revision	Verified by
Confidence, emotional state, risk level, expected outcome and decision date editable after the outcome	Edits rejected with 409; condition and update atomic	TC26, TC34–TC38
Outcome may be dated before the decision was logged	Rejected with 400	TC39
Two concurrent submissions store two outcomes	Atomic claim and unique index	TC40
Six report fields written unescaped	All user text HTML-escaped; restrictive content policy	TC31, TC41
One outcome query per decision	One joined query	statement counts, latency, load
A snapshot on every analytics request	One snapshot per user and day	TC43
Confidence calibration blind to opposite errors	Per-confidence-level error E_c	TC44, MR2b, power
Risk calibration ignores confidence	Per-risk-level error E_r	TC45, MR1–MR3, power
Warning words matched as substrings, negations counted	Whole words, inflections, negation window	TC46, MR7–MR9
No guidance on history length	Provisional flag below 20 decisions	power analysis
14 dependency advisories, 2 critical	Patched framework and libraries; unused PDF library removed	`npm audit`

TC, functional test case (Section 4.2); MR, metamorphic relation (Table 3); power, detection-power simulation (Section 4.4.2).

The dependency audit found 14 advisories in the baseline production dependencies, two of them critical (in the framework and in an unused PDF library) and one a high-severity SQL-injection advisory in the ORM's handling of identifiers. The revised implementation updated the framework and the affected libraries and removed the unused one; the audit then reported none.

6.2 RQ2: Performance

In the baseline, each analytics request issued one statement for the user's decisions, one per decision for its outcome and one to insert the snapshot: 22 statements at 20 decisions, 102 at 100 and 502 at 500. The dashboard followed the same pattern. The revised implementation issued 2 statements per analytics request and 3 per dashboard request at every history length. Median analytics latency in the baseline grew almost linearly, from 20.1 ms at 10 decisions to 924.1 ms at 1,000, while the revised implementation took 9.8 ms at 10 decisions and 36.7 ms at 1,000, 96% lower at 1,000 decisions (Fig. 3a). The dashboard page fell by 92% at 1,000 decisions. Every other endpoint answered within 12 ms at the median in both implementations, except registration and login, whose 0.46–0.48 s is spent almost entirely in bcrypt at cost factor 12 [36].

The gap widens with distance to the database (Fig. 3b). Each additional millisecond of

Table 5. Functional and safeguard tests passed, by area

Area	Tests	Passed, baseline	Passed, revised
Authentication	9	9	9
Route protection	4	4	4
Access control	4	4	4
Decision logging	8	8	8
Outcome recording	5	3	5
Prediction integrity	6	1	6
Analytics	6	2	6
Report export	4	2	4
Core tests (TC01–TC33)	33	31	33
Safeguard tests (TC34–TC46)	13	2	13
Total	46	33	46

TC, test case. Core tests TC01–TC33 and safeguard tests TC34–TC46 are described in Section 4.2.

Table 6. Analytics endpoint under ten concurrent clients for 20 seconds

Decisions	Implementation	Requests	Requests/s	Median (ms)	95th pct. (ms)	Errors
100	Baseline	196	9.8	1,025.0	1,236.9	0
100	Revised	2,584	129.2	71.7	109.6	0
500	Baseline	49	2.5	4,744.9	5,124.7	0
500	Revised	1,074	53.7	175.7	266.1	0

pct., percentile. Response times were measured at the client.

round-trip time added about 9.4 ms to a baseline analytics request at 100 decisions, whose 102 statements run at most ten at a time through the connection pool, against 2.1 ms for the revised implementation. Under ten concurrent clients (Table 6), the revised implementation sustained 13.2 times the throughput of the baseline at 100 decisions and 21.5 times at 500, where the baseline completed only 2.5 requests per second. Neither implementation returned an error. Snapshot growth was bounded too: after 31 analytics requests on one day, a user of the baseline held 31 snapshots and a user of the revised implementation 1.

6.3 RQ3: Score validity

Differential check. Every score that the running applications returned for the 100 simulated users and the worked example equalled the pure function of the same rules on the same inputs: 0 of 606 values differed for the baseline and 0 of 606 for the revised implementation. Under the baseline rules, the mean risk calibration score was 57.7 for every profile, as the rule's definition predicts when outcomes are shared. The offline results below therefore describe what the applications compute.

Metamorphic relations. Table 7 shows the violation rates. The baseline risk calibration score violated MR1, MR2a, MR2b and MR3 on every source history: since it never reads confidence, no distortion of confidence can lower it. Its confidence calibration score detected uniform shifts, but on balanced histories, where low- and high-risk errors cancel in the mean, it violated MR2b every time. Its emotional-bias score rose when "No regrets, not anxious." or "Fearless" was

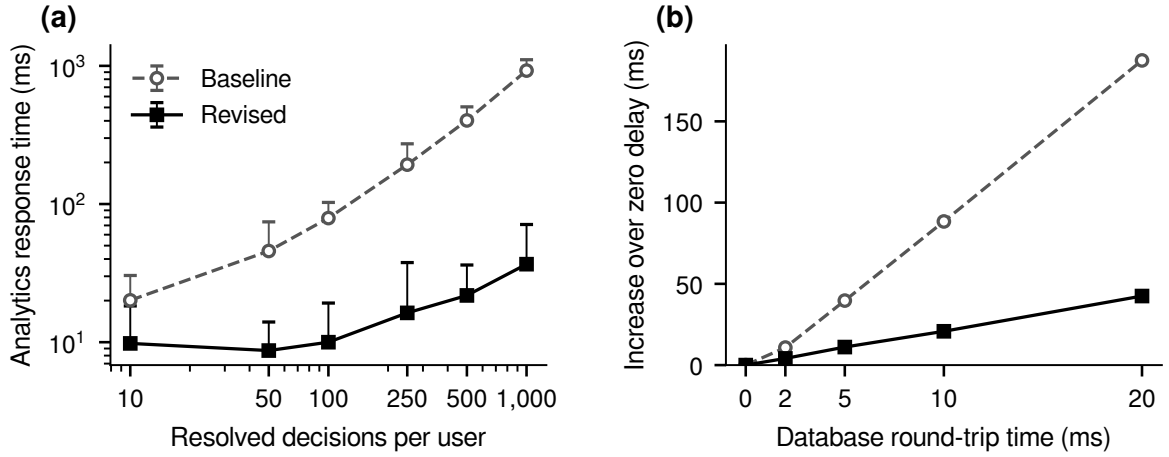


Figure 3. Analytics response time. (a) Median against the number of resolved decisions per user; whiskers extend to the 95th percentile of 30 requests. (b) Increase in median response time over its value with no added delay, against the emulated database round-trip time, for a user with 100 decisions (20 requests per point). Circles on dashed lines show the baseline and squares on solid lines the revised implementation; both axes in (a) are logarithmic

added. The revised implementation satisfied every relation, with one exception: in 0.1% of balanced sources, the opposite errors landed on the same confidence level and cancelled within that group, so the confidence calibration score did not fall. The risk calibration score, which groups by risk level, caught all of those cases. Both sets of rules satisfied the invariances MR4–MR6.

Simulated users. Table 8 shows the noisy-reporter simulation under both sets of rules. In the baseline the inconsistent profile, whose users misjudged every low- and high-risk decision by three steps, received the same confidence calibration score as the calibrated profile (98.2 and 98.2), and risk calibration did not vary at all. In the revised implementation the inconsistent profile scored 82.0 for confidence calibration and 79.0 for risk calibration, against 93.3 and 97.0 for the calibrated profile. The calibrated profile's revised confidence calibration score of 93.3 rather than 100 is itself informative: that profile adds case-to-case noise to its confidence, which a reliability diagram correctly shows as overconfidence at the extremes, while the baseline comparison of means could not see it.

Figure 4 shows detection power by history length. The baseline risk calibration score had an AUC of 0.50 for every profile and length, no better than chance. Its confidence calibration score never exceeded an AUC of about 0.6 for inconsistent users, however long the history, and was little better for noisy users. The revised risk calibration score separated ± 2 inconsistent users from calibrated ones with an AUC of 0.97 at 10 decisions and 1.00 at 20, and its confidence calibration score detected noisy users (0.97 at 20). Every two-step profile was flagged with at least 80% sensitivity at 95% specificity by 20 decisions; the revised implementation uses this threshold to mark scores as provisional. The revision has a cost: for a uniform bias of one step, the baseline comparison of means is the more efficient test, reaching 82% sensitivity at 20 decisions against 65% for the revised better score, because grouping spends information on detecting patterns that a uniform shift does not have. The revised implementation still reports the signed mean gap alongside the scores, so that information is not lost to the user.

Table 7. Share of 1,000 source histories on which each metamorphic relation was violated (%), by rules and score[†]

	Baseline rules				Revised rules			
Relation	DA	CC	RC	EB	DA	CC	RC	EB
MR1	–	0	100	–	–	0	0	–
MR2a	–	14.2	100	–	–	0	0	–
MR2b	–	100	100	–	–	0.1	0	–
MR3	–	0	100	–	–	0	0	–
MR4	0	0	0	0	0	0	0	0
MR5	0	0	0	0	0	0	0	0
MR6	0	0	0	–	0	0	0	–
MR7	–	–	–	100	–	–	–	0
MR8	–	–	–	100	–	–	–	0
MR9	–	–	–	0	–	–	–	0

[†]CC, confidence calibration; RC, risk calibration; DA, decision accuracy; EB, emotional bias. A dash means the relation does not apply to the score.

Table 8. Noisy-reporter simulation: mean (SD) over 25 users of 40 decisions per profile. All profiles share identical outcomes

	Mean gap	Accuracy	Baseline rules		Revised rules	
Profile	(steps)	(both)	CC	RC	CC	RC
Calibrated	0.0	86.2 (1.8)	98.2 (1.4)	57.7 (2.9)	93.3 (1.6)	97.0 (1.4)
Overconfident	1.9	79.2 (2.4)	81.4 (2.5)	57.7 (2.9)	81.1 (2.4)	81.4 (2.5)
Underconfident	−1.9	78.4 (1.9)	80.6 (2.4)	57.7 (2.9)	80.3 (2.4)	80.6 (2.4)
Inconsistent	−0.1	75.1 (2.1)	98.2 (1.8)	57.7 (2.9)	82.0 (3.4)	79.0 (2.0)

CC, confidence calibration; RC, risk calibration; SD, standard deviation. Mean gap is mean confidence minus mean success rating; accuracy is identical under both sets of rules.

Real forecasts. The 39 superforecasters were well calibrated as a group (pooled ECE 0.019, Brier score 0.110), but individuals varied (Fig. 5). Table 9 shows how each score related to the reference measures. The revised confidence calibration score correlated strongly with ECE ($\rho = -0.91$) and with Murphy's reliability term (-0.80); the baseline correlated only moderately (-0.42, -0.47), and almost perfectly with the absolute calibration-in-the-large (-0.96), which is all it measures. The revised risk calibration score correlated with the domain-conditional calibration error at -0.96. The baseline showed no relation to it (-0.08, interval spanning zero) and instead tracked how often each forecaster's questions resolved positively (0.51): it measures how things turned out, not calibration.

The controlled distortions of the real forecasts (Table 10) tell the same story. The baseline risk calibration score did not change for any forecaster under any distortion. Under uniform shifts, both implementations' confidence calibration scores agreed with the reference for most forecasters, the baseline slightly more often under underconfidence (90% against 87%), in line with the power analysis. Under opposite errors by domain, the revised risk calibration score moved in the right direction for 92% of forecasters and the baseline confidence calibration score for 69%. Doubling the log-odds was the hardest distortion for each of the other scores, probably because many superforecasters already state extreme probabilities, which doubling changes little on a ten-point scale; the revised confidence calibration score still agreed with the reference for 62% of forecasters against 28% for the baseline. With fewer forecasts the scores were noisier: on random subsets of 20 forecasts, the revised confidence calibration score

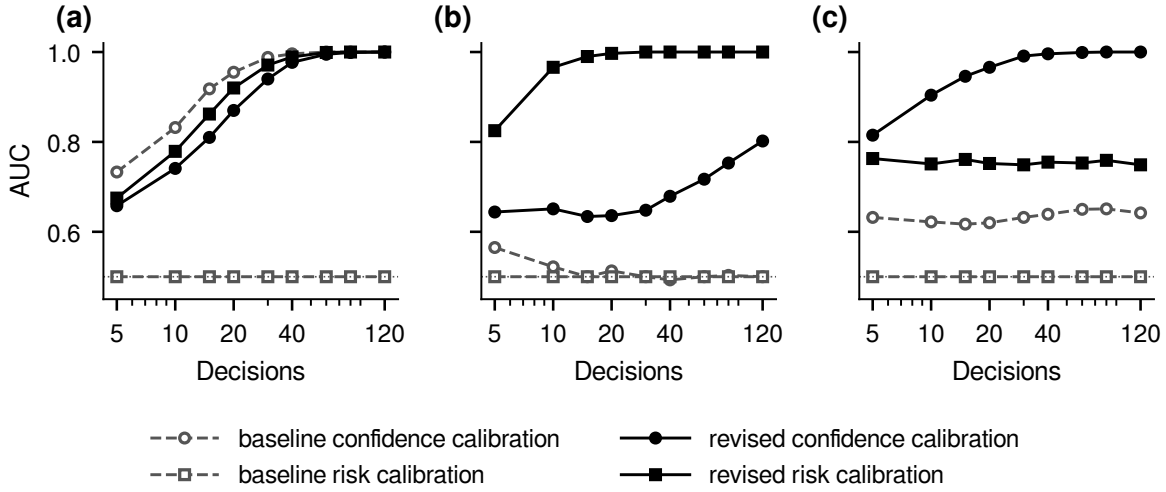


Figure 4. Detection power against history length: area under the ROC curve for separating 1,000 miscalibrated from 1,000 calibrated synthetic users with identical outcomes, for (a) a uniform one-step overconfidence, (b) ± 2 steps of risk-dependent inconsistency and (c) case-to-case noise with standard deviation 2. Open markers on dashed lines show baseline scores and filled markers on solid lines revised scores; circles mark confidence calibration and squares risk calibration. An AUC of 0.5 is chance

correlated with the full-history ECE at -0.36, rising to -0.61 at 80; for the baseline the figures were -0.15 and -0.28. With binary outcomes each forecast carries less information than a 1–10 rating, so real histories need more records than the simulation suggests.

6.4 RQ4: Test adequacy

Stryker generated 200 mutants of the revised scoring module (Table 11). The worked-example check killed 66.5% of them. Adding the metamorphic tests raised this to 73.5%, and adding the property-based tests to 76.0%. Example-based unit tests written in response to the surviving mutants, for instance one per negation word and per inflection, raised the score to 97.5%. The five mutants that survived all four sets alter the tokenizer in ways that cannot change any result, such as allowing empty tokens, and are equivalent. The example-based tests alone killed the same mutants as the full set, so by this measure the metamorphic and property-based tests added no fault detection once thorough examples existed.

That measure misses what the generated tests did contribute. During development, a property-based test of order invariance found that an earlier draft of the revised module produced calibration errors that depended on the order of decisions in their last floating-point digits, enough to change a reported score whenever an error fell on a rounding boundary; the module now computes every group statistic from integer sums. Another property, that the over- and under-confidence rates sum to at most 100, failed because the two rates are rounded separately, as in the baseline; the property was corrected to allow one point of rounding. More importantly, the metamorphic relations detect faults that mutation analysis cannot measure: faults in the specification. Every mutant of the revised module is a variant of a valid rule, whereas the baseline risk calibration rule was invalid as written and would have passed any test derived from its own formula, including the worked example.

Table 9. Spearman correlation between application scores and reference calibration measures across 39 superforecasters, with 95% bootstrap confidence intervals. Because the scores are high when calibration is good and the reference errors are high when it is bad, a valid score correlates negatively with its reference error

Score	Reference measure	ρ	95% CI
Baseline confidence calibration	ECE	−0.42	[−0.69, −0.10]
Revised confidence calibration	ECE	−0.91	[−0.96, −0.81]
Baseline confidence calibration	reliability	−0.47	[−0.70, −0.15]
Revised confidence calibration	reliability	−0.80	[−0.90, −0.63]
Baseline confidence calibration	|calibration-in-the-large|	−0.96	[−0.98, −0.91]
Baseline risk calibration	domain calibration error	−0.08	[−0.38, 0.23]
Revised risk calibration	domain calibration error	−0.96	[−0.98, −0.91]
Baseline risk calibration	outcome base rate	0.51	[0.21, 0.72]
Revised risk calibration	outcome base rate	−0.26	[−0.52, 0.06]

ρ, Spearman rank correlation; CI, confidence interval; ECE, expected calibration error.

Table 10. Share of 39 forecasters for whom a score moved in the direction indicated by the reference measure when their real forecasts were distorted (%). Reference: ECE for D1, D2 and D4; domain-conditional calibration error for D3

		Baseline rules		Revised rules	
	Distortion	CC	RC	CC	RC
D1	Uniform overconfidence: $p+0.2$	95	0	100	97
D2	Uniform underconfidence: $p-0.2$	90	0	87	85
D3	Opposite errors by domain: market −0.2, event +0.2	69	0	85	92
D4	Extremised: logit doubled	28	0	62	33

CC, confidence calibration; RC, risk calibration; p, stated probability; ECE, expected calibration error.

7 Discussion

A score's name is a specification. The two invalid baseline scores were not coding errors. Each implemented its formula correctly, and a worked example confirmed it to the last digit. What failed was the formula's relation to its name. Metamorphic relations written from the name (more overconfidence must lower a calibration score, and confidence must matter to a calibration score) exposed both defects on every input, needed no ground truth, and took a few lines each to state. We think this is the cheapest check a developer of any self-tracking score can run, and it belongs alongside the code's unit tests rather than in a later study.

Averages hide inconsistency, and grouping has a price. The baseline confidence calibration score was a correct measure of calibration-in-the-large and nothing more. Kahneman et al. describe the case-to-case variability it could not see as noise and argue that it damages judgment as much as bias does [37]. Grouping by confidence and by risk level exposes inconsistency and

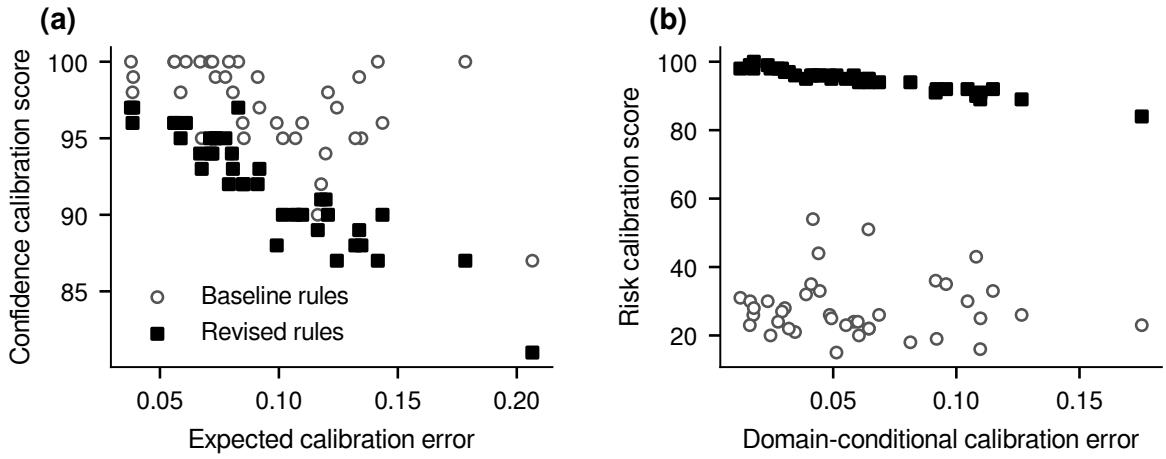


Figure 5. Application scores against reference measures for 39 superforecasters. (a) Confidence calibration against expected calibration error (ECE); (b) risk calibration against domain-conditional calibration error. Open circles show the baseline rules and filled squares the revised rules; each forecaster appears once under each

Table 11. Mutation analysis of the revised scoring module (200 mutants)

Test set	Killed	Survived	Score (%)
Worked-example check	133	67	66.5
+ metamorphic tests	147	53	73.5
+ property-based tests	152	48	76.0
+ example-based unit tests (full suite)	195	5	97.5
Worked-example check + example-based unit tests only	195	5	97.5

Score, killed mutants as a percentage of all mutants.

noise, and on real forecasts it brought the scores in line with the measures forecasters are judged by. It costs some sensitivity to a small uniform bias, and with the ten confidence levels as bins, opposite errors that fall on the same level can still cancel; reporting the signed mean gap and the per-risk gaps next to the scores covers both cases. A score that summarises calibration in one number will always lose something, and the application now says which number to read for which question.

How many decisions are enough. Self-tracking applications show scores from the first record. The power analysis gives a defensible threshold: with 20 resolved decisions every two-step miscalibration is flagged reliably in simulation. The replay is a warning that this is optimistic for binary outcomes, where 20 records gave only a moderate correlation with the full-history reference. Marking early scores as provisional is cheap and honest. A person who logs a few consequential decisions a month may need most of a year to reach the threshold, which is worth stating plainly in the interface.

The record needs the same protection as the score. A prospective decision log protects against hindsight only if the prediction cannot be revised [16], and the interface alone did not enforce this. The race condition is a reminder that check-then-act code in web applications is unsafe under concurrency even when every individual request is handled correctly [28]; a

conditional update plus a database constraint closed it. The N+1 query pattern cost little on a local database and much more at realistic round-trip times, which is where a hosted deployment runs.

What the scores still cannot tell. The success rating mixes judgment with luck, so every score built on it inherits outcome bias [17]. The optional decision-quality rating in the revised implementation lets a user separate the two, but no score uses it yet. The emotional-bias score remains a twelve-word indicator; the revisions made its matching correct, not its construct valid, and validating it would need affect measures that only a study with participants can provide.

8 Threats to validity

Construct validity. The simulation encodes one model of how confidence and outcomes relate; the conclusions about detection power hold under that model, and the real-forecast replay was added because of it. In the replay, probabilities and binary outcomes were mapped to the application's 1–10 scales and question domain stood in for risk level. Forecasting questions are not personal decisions, and superforecasters are far better calibrated than most people, which compresses the range of scores and makes the correlations, if anything, conservative. The reference measures are standard but are themselves estimates subject to binning choices.

Internal validity. All timings come from one machine with two virtual CPUs, a local database and an emulated network; absolute values will differ in production. Timings also vary between runs on the same hardware, which is why both implementations were measured in the same run. The tests and relations were written by the developers of the system, and some tests were written after the defects were known; mutation analysis and the replay against independent reference measures limit, but do not remove, that bias. The use of a language model to write evaluation code (Section 4.6) is mitigated by the differential check and by the published scripts and raw results.

External validity. No real users of the application took part, so we cannot say how people will use the scores or whether seeing them improves calibration, as feedback does for forecasters [9]. The study covers one application. The functional tests cover common cases rather than the full attack surface, and no independent security audit was performed.

Conclusion validity. The replay rests on 39 forecasters, and the bootstrap intervals in Table 9 are correspondingly wide; the conclusions drawn from it concern clear differences between the baseline and revised rules, not precise values. Simulation results use 1,000 users per cell and fixed seeds and are exactly reproducible.

9 Conclusion

Digital Memory Vault records personal decisions as predictions and scores them against their outcomes. Testing the code that computed its scores was not enough: two of the four scores computed their formulas correctly and still did not measure what their names claimed. Metamorphic relations derived from the names exposed both defects without any ground truth, simulation with known behaviour showed what each score could detect and after how many decisions, and real forecasts confirmed that the revised scores agree with established calibration measures where the baseline scores did not. The same evaluation found, and verified revisions for, editable predictions, a race that duplicated outcomes, unescaped report fields, a query pattern whose cost grew with every decision, and vulnerable dependencies. Scoring rules in self-

tracking software are measurement instruments, and they can and should be tested as rigorously as the code that computes them. The next steps are a study with real users to learn whether the revised scores improve their calibration over time, and validation of the emotional-bias indicator against measured affect.

References

[1] Barr ET, Harman M, McMinn P, Shahbaz M, Yoo S. The oracle problem in software testing: a survey. *IEEE Transactions on Software Engineering* 2015; **41**(5): 507–525. DOI: 10.1109/TSE.2014.2372785.

[2] Chen TY, Cheung SC, Yiu SM. Metamorphic testing: a new approach for generating next test cases. *Technical Report HKUST-CS98-01*, Department of Computer Science, Hong Kong University of Science and Technology: Hong Kong, 1998. Reprinted as arXiv:2002.12543, 2020.

[3] Chen TY, Kuo FC, Liu H, Poon PL, Towey D, Tse TH, Zhou ZQ. Metamorphic testing: a review of challenges and opportunities. *ACM Computing Surveys* 2018; **51**(1): 1–27. DOI: 10.1145/3143561.

[4] Segura S, Fraser G, Sanchez AB, Ruiz-Cortés A. A survey on metamorphic testing. *IEEE Transactions on Software Engineering* 2016; **42**(9): 805–824. DOI: 10.1109/TSE.2016.2532875.

[5] Brier GW. Verification of forecasts expressed in terms of probability. *Monthly Weather Review* 1950; **78**(1): 1–3. DOI: 10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.

[6] Murphy AH. A new vector partition of the probability score. *Journal of Applied Meteorology* 1973; **12**(4): 595–600. DOI: 10.1175/1520-0450(1973)012<0595:ANVPOT>2.0.CO;2.

[7] Lichtenstein S, Fischhoff B, Phillips LD. Calibration of probabilities: the state of the art to 1980. In *Judgment under Uncertainty: Heuristics and Biases*, Kahneman D, Slovic P, Tversky A (eds.). Cambridge University Press: Cambridge, 1982; 306–334.

[8] Moore DA, Healy PJ. The trouble with overconfidence. *Psychological Review* 2008; **115**(2): 502–517. DOI: 10.1037/0033-295X.115.2.502.

[9] Moore DA, Swift SA, Minster A, Mellers B, Ungar L, Tetlock P, Yang HHJ, Tenney ER. Confidence calibration in a multiyear geopolitical forecasting competition. *Management Science* 2017; **63**(11): 3552–3565. DOI: 10.1287/mnsc.2016.2525.

[10] Mellers B, Ungar L, Baron J, Ramos J, Gurcay B, Fincher K, Scott SE, Moore D, Atanasov P, Swift SA, Murray T, Stone E, Tetlock PE. Psychological strategies for winning a geopolitical forecasting tournament. *Psychological Science* 2014; **25**(5): 1106–1115. DOI: 10.1177/0956797614524255.

[11] Pakdaman Naeini M, Cooper GF, Hauskrecht M. Obtaining well calibrated probabilities using Bayesian binning. In *Proceedings of the 29th AAAI Conference on Artificial Intelligence*. AAAI Press: Austin, 2015; 2901–2907. DOI: 10.1609/aaai.v29i1.9602.

[12] Guo C, Pleiss G, Sun Y, Weinberger KQ. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, PMLR 70. PMLR: Sydney, 2017; 1321–1330.

[13] Kleinberg J, Mullainathan S, Raghavan M. Inherent trade-offs in the fair determination of risk scores. In *8th Innovations in Theoretical Computer Science Conference (ITCS 2017)*, LIPIcs 67. Schloss Dagstuhl: Dagstuhl, 2017; 43:1–43:23. DOI: 10.4230/LIPIcs.ITCS.2017.43.

[14] Li I, Dey A, Forlizzi J. A stage-based model of personal informatics systems. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM: New York, 2010; 557–566. DOI: 10.1145/1753326.1753409.

[15] Epstein DA, Caldeira C, Figueiredo MC, Lu X, Silva LM, Williams L, Lee JH, Li Q, Ahuja S, Chen Q, Dowlatyari P, Hilby C, Sultana S, Eikey EV, Chen Y. Mapping and taking stock of the personal informatics literature. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 2020; **4**(4): 126. DOI: 10.1145/3432231.

[16] Fischhoff B. Hindsight $\neq$ foresight: the effect of outcome knowledge on judgment under uncertainty. *Journal of Experimental Psychology: Human Perception and Performance* 1975; **1**(3): 288–299. DOI: 10.1037/0096-1523.1.3.288.

[17] Baron J, Hershey JC. Outcome bias in decision evaluation. *Journal of Personality and Social Psychology* 1988; **54**(4): 569–579. DOI: 10.1037/0022-3514.54.4.569.

[18] Lerner JS, Li Y, Valdesolo P, Kassam KS. Emotion and decision making. *Annual Review of Psychology* 2015; **66**: 799–823. DOI: 10.1146/annurev-psych-010213-115043.

[19] Kitchenham B, Pfleeger SL, Fenton N. Towards a framework for software measurement validation. *IEEE Transactions on Software Engineering* 1995; **21**(12): 929–944. DOI: 10.1109/32.489070.

[20] Briand LC, Morasca S, Basili VR. Property-based software engineering measurement. *IEEE Transactions on Software Engineering* 1996; **22**(1): 68–86. DOI: 10.1109/32.481535.

[21] Kaner C, Bond WP. Software engineering metrics: what do they measure and how do we know? In *Proceedings of the 10th International Software Metrics Symposium (METRICS 2004)*. IEEE: Chicago, 2004.

[22] Xie X, Ho JWK, Murphy C, Kaiser G, Xu B, Chen TY. Testing and validating machine learning classifiers by metamorphic testing. *Journal of Systems and Software* 2011; **84**(4): 544–558. DOI: 10.1016/j.jss.2010.11.920.

[23] Claessen K, Hughes J. QuickCheck: a lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*. ACM: New York, 2000; 268–279. DOI: 10.1145/351240.351266.

[24] Jia Y, Harman M. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering* 2011; **37**(5): 649–678. DOI: 10.1109/TSE.2010.62.

[25] Papadakis M, Kintis M, Zhang J, Jia Y, Le Traon Y, Harman M. Mutation testing advances: an analysis and survey. *Advances in Computers* 2019; **112**: 275–378. DOI: 10.1016/bs.adcom.2018.03.015.

[26] Chen TH, Shang W, Jiang ZM, Hassan AE, Nasser M, Flora P. Detecting performance antipatterns for applications developed using object-relational mapping. In *Proceedings of the 36th International Conference on Software Engineering*. ACM: New York, 2014; 1001–1012. DOI: 10.1145/2568225.2568259.

[27] Yang J, Yan C, Subramaniam P, Lu S, Cheung A. How not to structure your database-backed web applications: a study of performance bugs in the wild. In *Proceedings of the 40th International Conference on Software Engineering*. ACM: New York, 2018; 800–810. DOI: 10.1145/3180155.3180194.

[28] Warszawski T, Bailis P. ACIDRain: concurrency-related attacks on database-backed web applications. In *Proceedings of the 2017 ACM International Conference on Management of Data*. ACM: New York, 2017; 5–20. DOI: 10.1145/3035918.3064037.

[29] OWASP Foundation. OWASP Top 10:2021. `https://owasp.org/Top10/` [accessed 12 October 2026].

[30] Fawcett T. An introduction to ROC analysis. *Pattern Recognition Letters* 2006; **27**(8): 861–874. DOI: 10.1016/j.patrec.2005.10.010.

[31] Karger E, Bastani H, Yueh-Han C, Jacobs Z, Halawi D, Zhang F, Tetlock PE. ForecastBench: a dynamic benchmark of AI forecasting capabilities. In *Proceedings of the 13th International Conference on Learning Representations (ICLR 2025)*. Singapore, 2025.

[32] Efron B, Tibshirani RJ. *An Introduction to the Bootstrap*. Chapman & Hall: New York, 1993.

[33] Stryker Mutator contributors. StrykerJS: mutation testing for JavaScript and TypeScript, version 9.6. `https://stryker-mutator.io` [accessed 12 October 2026].

[34] Dubien N. fast-check: property-based testing framework for JavaScript and TypeScript, version 4.10. `https://fast-check.dev` [accessed 12 October 2026].

[35] Tversky A, Kahneman D. Judgment under uncertainty: heuristics and biases. *Science* 1974; **185**(4157): 1124–1131. DOI: 10.1126/science.185.4157.1124.

[36] Provos N, Mazières D. A future-adaptable password scheme. In *Proceedings of the 1999 USENIX Annual Technical Conference, FREENIX Track*. USENIX Association: Monterey, 1999; 81–91.

[37] Kahneman D, Sibony O, Sunstein CR. *Noise: A Flaw in Human Judgment*. Little, Brown Spark: New York, 2021.

Acknowledgments

The author thanks the Forecasting Research Institute for publishing the ForecastBench datasets under an open licence.

Supporting information

Additional supporting information may be found online in the Supporting Information section at the end of this article. **Supporting Information S1** (zip archive): the source code of the baseline and revised implementations, the evaluation kit, the raw results of the run reported here, every number in the paper as generated macros, and the derived real-forecast dataset.